CAPSTONE

WsCube Tech
Upskilling Bharat

New York Stock Exchange

U.S. EQUITY MARKET INTELLIGENCE & PORTFOLIO ANALYTICS

PRESENTED BY : DIVYA SHARMA

Introduction

NYSE Financial Analysis & Forecasting

THIS PROJECT FOCUSES ON TRANSFORMING RAW MARKET DATA INTO A STRATEGIC INVESTMENT FRAMEWORK.

OBJECTIVE:
TO BRIDGE THE GAP BETWEEN HISTORICAL FINANCIAL HEALTH AND FUTURE MARKET PERFORMANCE.

GOAL:
TO PROVIDE A DATA-DRIVEN FRAMEWORK FOR RISK MANAGEMENT AND STRATEGIC INVESTMENT DECISION-MAKING.

DATASET USED

Fundamentals

Company Financial Health

Core financial statement data revealing how well a company is performing operationally and financially.

- Revenue
- Net Income
- EPS
- Total Assets
- Liabilities
- Cash Flow
- ROE / ROA
- Profit Margins

SEC Filings

Regulatory & Filing Data

Official disclosures submitted to the U.S. Securities & Exchange Commission covering company reporting and compliance.

- Company Name
- Ticker Symbol
- Filing Date
- Report Type (10-K/10-Q)
- Shares Outstanding
- Fiscal Year

Prices (Split Adjusted)

Historical Stock Prices

Daily stock price data adjusted for corporate splits, ensuring accurate historical trend and performance analysis.

- Date
- Open
- High
- Low
- Close
- Volume
- Adj. Close Price

Part 1 : SQL Analysis

1. *Calculate the average net income per year for each ticker symbol and identify the top 3 companies with the highest average net income.*

Syntax

```sql
SELECT
    ticker_symbol,
    AVG(net_income) AS avg_net_income
FROM fundamentals
GROUP BY ticker_symbol
ORDER BY avg_net_income DESC
LIMIT 3;
```

Output

ticker_symbol	avg_net_income
AAPL	43907000320
XOM	31532499456
WFC	21681500160

PURPOSE

This query identifies which companies consistently generate the highest profit.

CONCEPT USED

- AVG = average across years
- GROUP BY = one row per company
- ORDER BY = highest profit first

2. *Find the year-over-year growth percentage of total revenue for a specific company (ticker_symbol = 'AAL') for the years available in the data.*

Syntax

```sql
with t as
(SELECT
    Ticker_Symbol,
    For_Year,
    Total_Revenue,
    LAG(Total_Revenue) OVER (ORDER BY For_Year) as Previous_year
FROM
    fundamentals
where Ticker_Symbol = 'AAL')

select Ticker_Symbol,
    For_Year,
    Total_Revenue,
    Previous_year,
   round(((Total_Revenue - Previous_year)/ Previous_year)*100,2) as YOY_Growth
    from t
    ;
```

Output

ticker_symbol	period_ending	total_revenue	yoy_growth_percentage
AAL	2012-12-31	24855000000	NULL
AAL	2013-12-31	26743000000	7.6
AAL	2014-12-31	42650000000	59.48
AAL	2015-12-31	40990000000	-3.89

PURPOSE

This analysis tracks revenue momentum for a specific firm, providing insights into its expansion or contraction patterns over time.
Positive growth: Market Expansion , Negative growth: Industry Stress

CONCEPT USED

LAG() window function
to compare current year with previous year.

3. *Identify the companies and years where the profit margin was negative, and list those records along with the net income and total revenue.*

Syntax

```sql
SELECT
    For_year,
    ticker_symbol,
    Profit_Margin,
    Net_Income,
    Total_Revenue
FROM
    fundamentals
where Profit_Margin < 0;
```

Output

For_year	ticker_symbol	Profit_Margin	Net_Income	Total_Revenue

PURPOSE

This query identifies which companies lost money despite revenue.
Negative margin = loss-making year.

CONCEPT USED

WHERE clause filter

4. *For each company, compute the average quick ratio over all years and find companies with an average quick ratio below 1, indicating potential liquidity issues.*

Syntax

```sql
SELECT
    ticker_symbol, AVG(Quick_Ratio) AS AVG
FROM
    fundamentals
GROUP BY Ticker_Symbol
HAVING AVG < 1;
```

Output

ticker_symbol	AVG

PURPOSE

We apply this query to detect companies with average quick ratio below 1, highlighting potential liquidity issues.
Quick ratio = short-term liquidity strength

CONCEPT USED

AVG()
GROUP BY
HAVING

5. *Using window functions, find the 3-year rolling average of research and development expenses (research_and_development) for each company.*

Syntax

```sql
SELECT
    Ticker_Symbol,
    For_Year,
    Research_and_Development,
    AVG(Research_and_Development) OVER (PARTITION BY Ticker_Symbol
        ORDER BY For_Year
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS Rolling_3Yr_Avg_RD
FROM fundamentals;
```

Output

Ticker_Symbol	For_Year	Research_and_Development	Rolling_3Yr_Avg_RD
AAL	2012	0	0
AAL	2013	0	0
AAL	2014	0	0
AAL	2015	0	0
AAP	2012	0	0
AAP	2013	0	0
AAP	2014	0	0
AAP	2015	0	0
AAPL	2013	4475000000	4474999808
AAPL	2014	6041000000	5257999872
AAPL	2015	8067000000	6194333184
AAPL	2016	10045000000	8050999808
ABBV	NULL	2778000000	2777999872
ABBV	2013	2855000000	2816499968
ABBV	2014	3297000000	2976666624

PURPOSE

Query Indicates trends in company investment in R&D over a rolling period.
Rising rolling average often means:

- product investment
- long-term expansion

CONCEPT USED

AVG() window function + ROWS BETWEEN

6. *Find companies where the capital expenditures have increased consecutively for at least 3 years.*

Syntax

```sql
SELECT *
FROM (
    SELECT
        Ticker_Symbol,
        For_Year,
        Capital_Expenditures,
        LAG(Capital_Expenditures,1) OVER (
            PARTITION BY Ticker_Symbol ORDER BY For_Year
        ) AS Prev1,
        LAG(Capital_Expenditures,2) OVER (
            PARTITION BY Ticker_Symbol ORDER BY For_Year
        ) AS Prev2
    FROM fundamentals
) as t
WHERE Prev2 < Prev1
AND Prev1 < Capital_Expenditures;
```

Output

Ticker_Symbol	For_Year	Capital_Expenditures	Prev1	Prev2
ABT	2014	-1077000000	-1145000000	-1795000000
ADBE	2014	-148332000	-188358000	-203805000
ADI	2016	-127397000	-153960000	-177913000
ADM	2014	-894000000	-913000000	-1240000000
AJG	2014	-81500000	-93600000	-217800000
ALB	2014	-110576000	-155346000	-280873000
APD	2015	-1265600000	-1362700000	-1524200000
APD	2016	-1055800000	-1265600000	-1362700000
BHI	2015	-965000000	-1791000000	-2085000000
BMY	2014	-526000000	-537000000	-548000000
BXP	2014	-410612000	-919735000	-1144450000
BXP	2015	-374664000	-410612000	-919735000
C	2014	-3386000000	-3490000000	-3604000000

PURPOSE

This identifies companies aggressively scaling their infrastructure and asset base, often a sign of planned future expansion.

CONCEPT USED

LAG() multiple times
Need previous 2 years

7. *List the top 5 years (period_ending) with the highest total liabilities summed over all companies.*

Syntax

```sql
SELECT
    For_Year,
    SUM(Total_Liabilities) AS Total_Liabilities_All
FROM fundamentals
GROUP BY For_Year
ORDER BY Total_Liabilities_All DESC
LIMIT 5;
```

Output

For_Year	Total_Liabilities_All
2014	19570675859424
2013	18444421682880
2015	17870560246336
2012	14886179725824
NULL	5886000952704

PURPOSE

Query indicates which years had highest overall corporate debt burden or when overall market liabilities were highest, pointing to financial risk periods

CONCEPT USED

GROUP BY + SUM() + ORDER BY + LIMIT

8. *For each company, find the year with the maximum net cash flow from operating activities (net_cash_flow_operating).*

Syntax

```sql
SELECT
    Ticker_Symbol,
    For_Year,
    Net_Cash_Flow_Operating
FROM (
    SELECT *,
        ROW_NUMBER() OVER(
            PARTITION BY Ticker_Symbol
            ORDER BY Net_Cash_Flow_Operating DESC
        ) AS rn
    FROM fundamentals
) t
WHERE rn = 1;
```

Output

Ticker_Symbol	For_Year	Net_Cash_Flow_Operating
AAL	2015	6249000000
AAP	2014	708991000
AAPL	2015	81266000000
ABBV	2015	7535000000
ABC	2015	3922230000
ABT	2012	9314000000
ADBE	NULL	2199730000
ADI	2016	1280890000
ADM	2013	5226000000
ADS	2015	1705840000
ADSK	2015	708100000
AEE	2015	2017000000
	NULL	

PURPOSE

Pinpoints the year a company generated the most cash from operations, showing operational efficiency.

CONCEPT USED

ROW_NUMBER() window function + PARTITION BY + filter rn=1

9. *Find the average earnings per share (EPS) for companies where the total equity exceeds 5 billion and the operating margin is greater than 10%.*

Syntax

```sql
SELECT
    Ticker_Symbol,
    AVG(Earnings_Per_Share) AS Avg_EPS
FROM fundamentals
WHERE Total_Equity > 5000000000
AND Operating_Margin > 0.10
GROUP BY Ticker_Symbol;
```

Output

Ticker_Symbol	Avg_EPS
AAL	11.390000343322754
AAPL	16.03749966621399
ABT	2.4600000083446503
ADBE	0.7933333317438761
ADI	0.6349999904632568
ADM	2.8200000127156577
AEE	0.5499999523162842
AEP	2.9933332602183023
AFL	6.340000033378601
AIG	4.37333329518636
AIZ	6.519999980926514
ALL	5.267499923706055

PURPOSE

Reveals which large, profitable companies provide strong shareholder returns via EPS or Among financially strong firms, what is average EPS?

CONCEPT USED

WHERE clause filter + AVG() + GROUP BY

10. *Determine the correlation (using MySQL approximate calculation) between research and development expenses and net income for a selected company (ticker_symbol = 'AAL').*

Syntax

```sql
SELECT
(COUNT(*) * SUM(Research_And_Development * Net_Income) -
SUM(Research_And_Development) * SUM(Net_Income))
/
SQRT((COUNT(*) * SUM(POWER(Research_And_Development,2))
-
POWER(SUM(Research_And_Development),2))
*
(COUNT(*) * SUM(POWER(Net_Income,2))
-
POWER(SUM(Net_Income),2))) AS Correlation
FROM fundamentals
WHERE Ticker_Symbol = 'AAL';
```

Output

Correlation
NULL

PURPOSE

It tells us whether increases in R&D spending are associated with higher (or lower) profits.

- Positive correlation → R&D investments tend to increase net income.
- Negative correlation → R&D investments may not be generating expected profit.

CONCEPT USED

Aggregate correlation formula using sums and powers

Part 2 : PowerBi Dashboards

Power BI

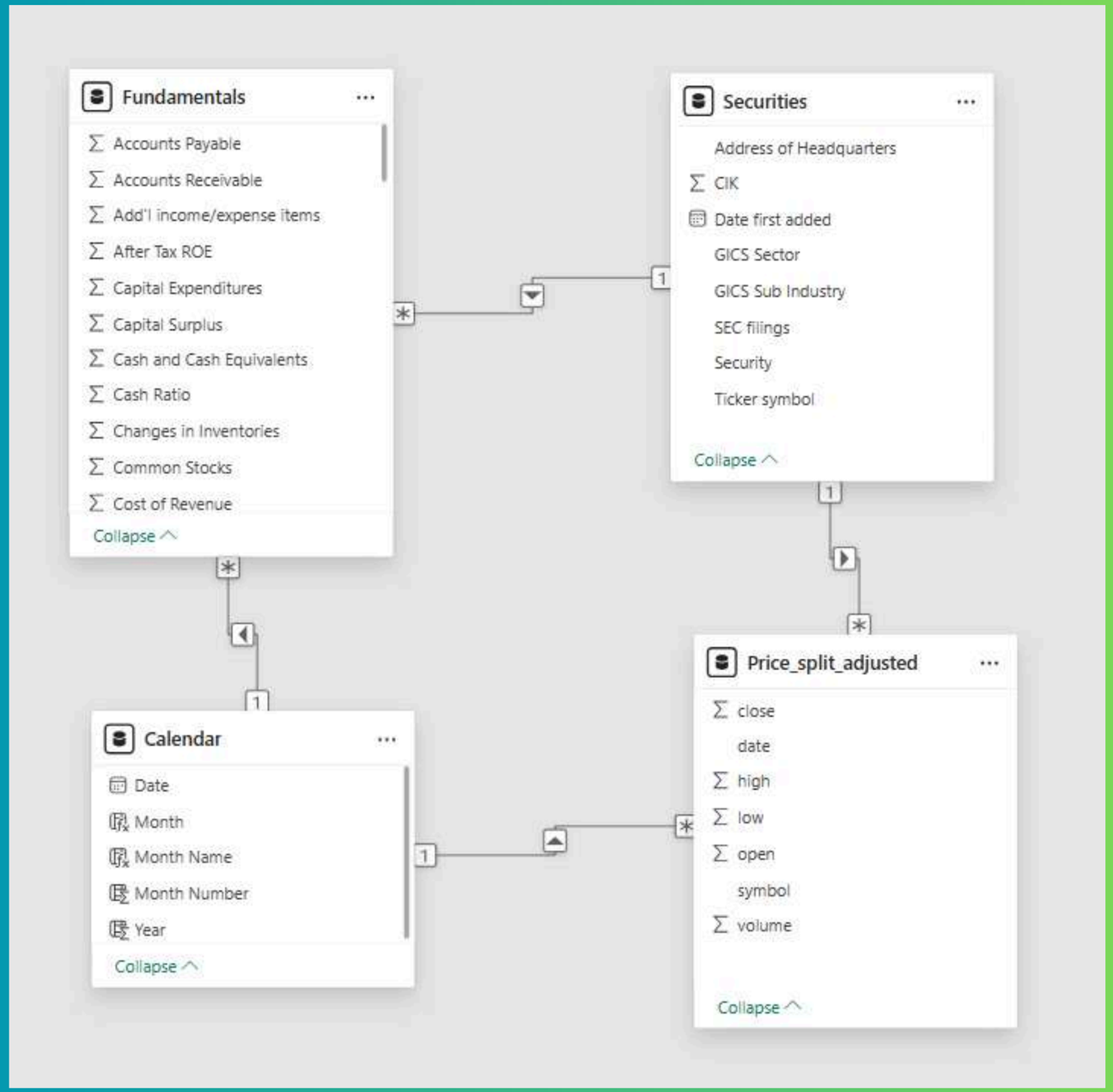
Data Modelling

DATA MODEL: STAR SCHEMA

CARDINALITY: ONE-TO-MANY

FACT TABLE -
- FUNDAMENTALS
- PRICE_SPLIT_ADJUSTED

DIMENSIONS-
- SECURITIES
- CALENDAR



Dashboard Questions Flow

1 Executive Market Overview

- Total companies analyzed
- Average net income
- Monthly volume trend
- MoM adjusted price growth

2 Financial Insights

- Top 10 companies by profit
- YoY revenue growth
- ROE vs net margin
- Debt/Equity comparison
- P/E trends

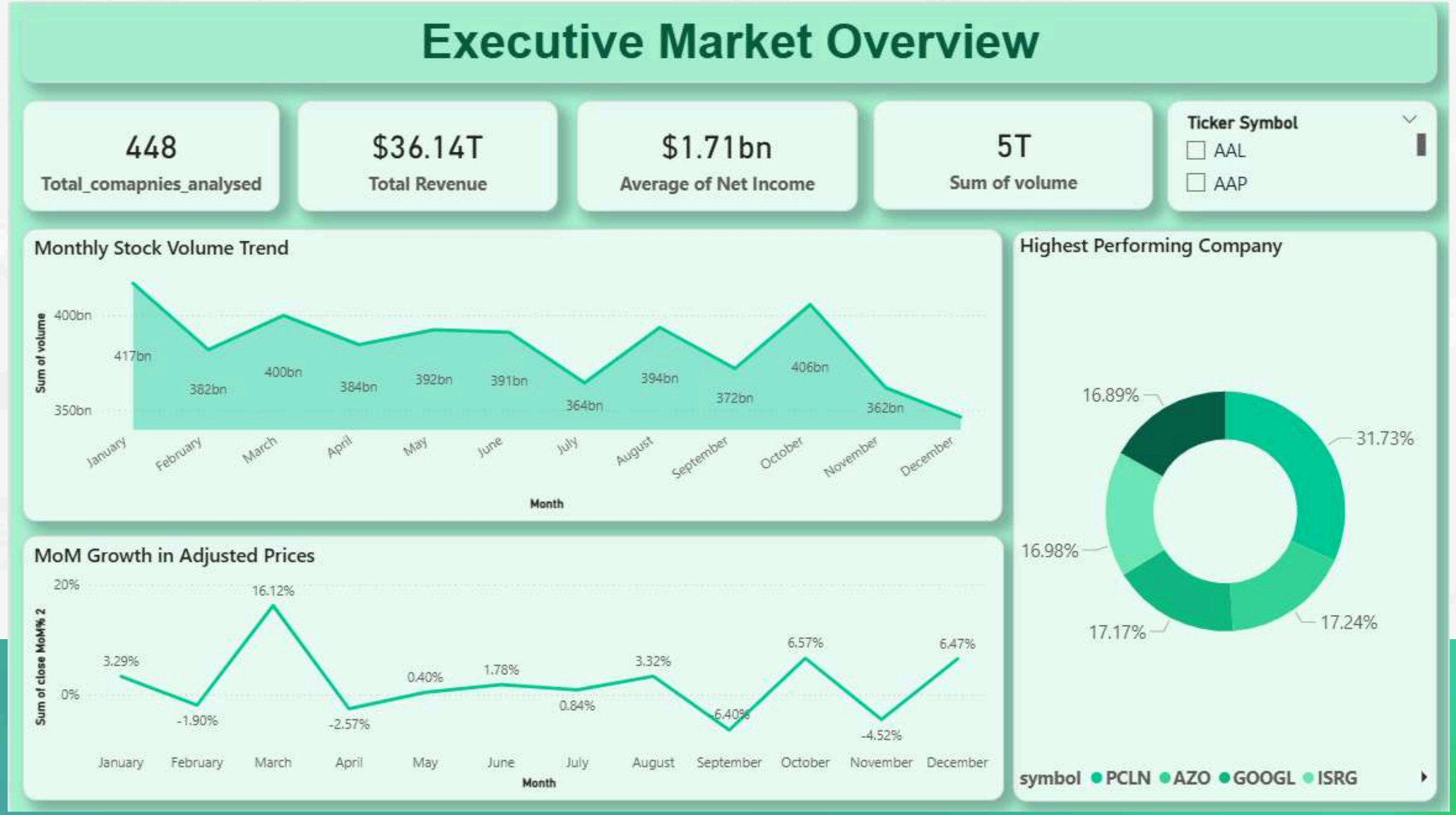
3 Price & Trading Performance

- Monthly adjusted close trend
- Volatility score
- Trading volume trend
- Stable stocks identification
- Price change tracking

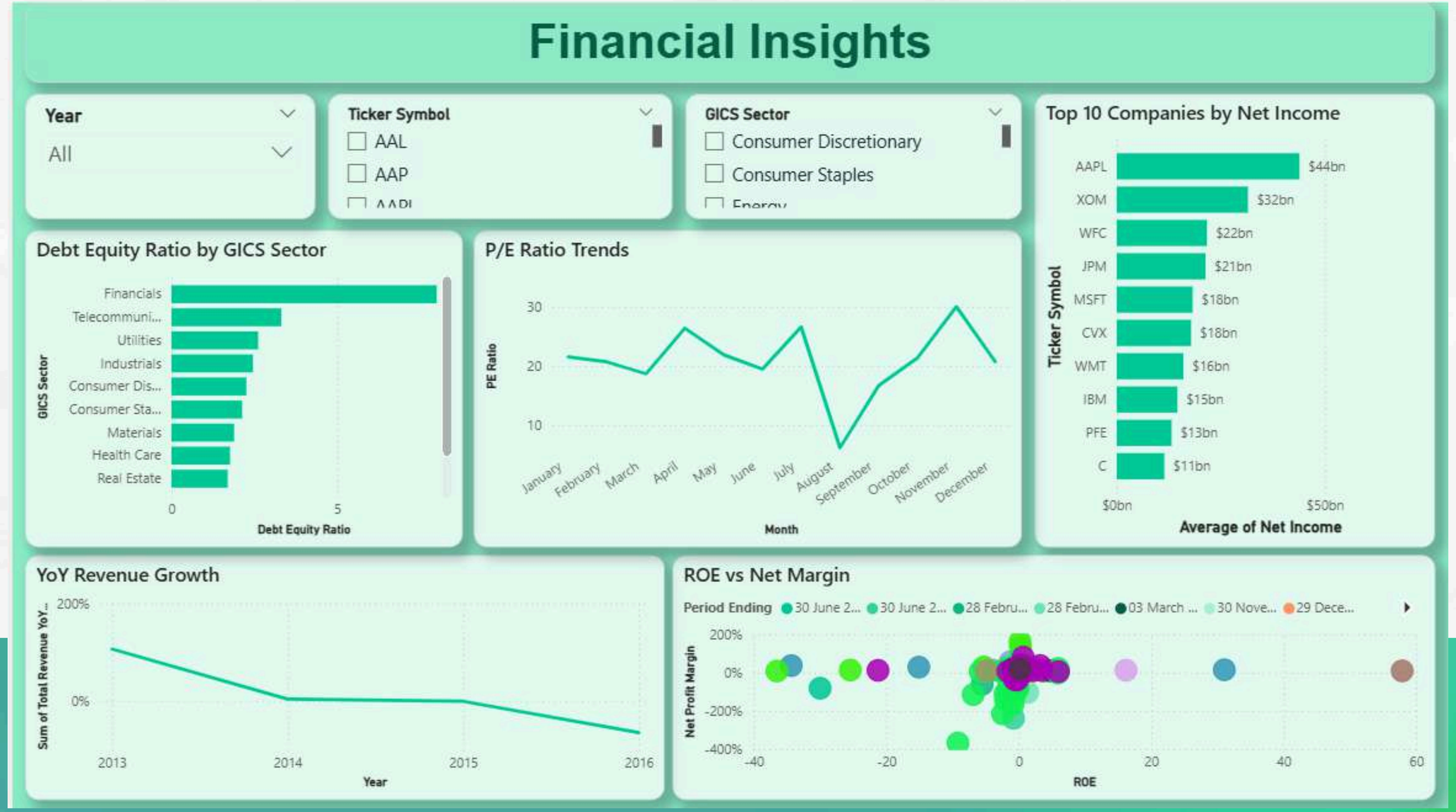
4 Sector Analytics

- Sector profit comparison
- Revenue trend by sector
- Profit margin leaders
- Debt burden by sector
- Market cap vs profitability

Page 1: Executive Market Overview

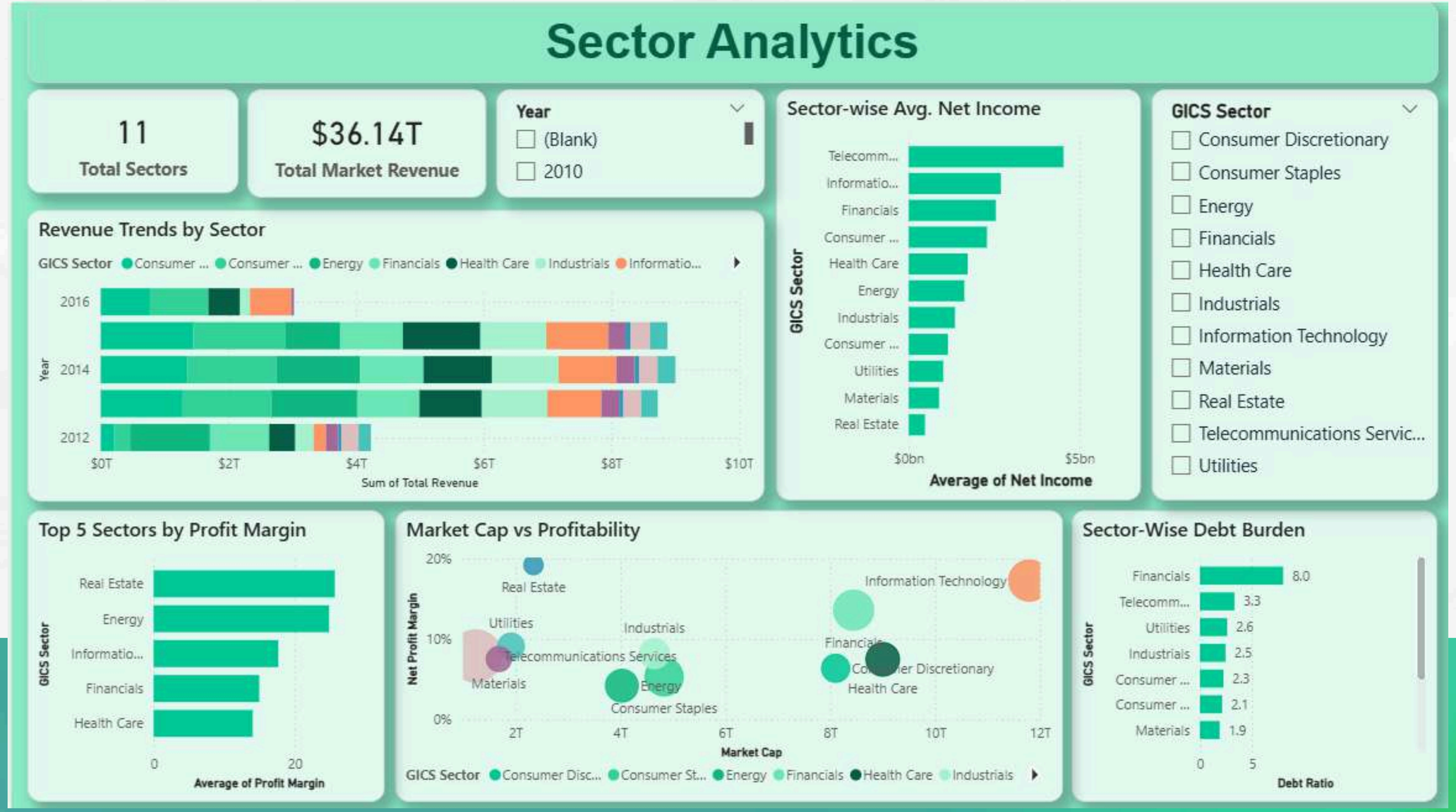


Page 2: Financial Insights



Page 3: Price & Trading Performance

Page 4: Sector Analytics



DASHBOARD SUMMARY NYSE ANALYSIS

01

PAGE 1

- This page gives an overall summary of the stock market data.
- Total companies analyzed shows how many companies are included in analysis.
- Average net income tells whether companies are earning profit on average.
- Monthly stock volume trend shows when buying and selling activity increased.
- MoM adjusted price growth helps understand monthly market movement.

02

PAGE 2

- This page explains company financial performance.
- Top 10 companies by net income shows highest profit-making companies.
- YoY revenue growth shows whether company sales increased each year.
- ROE vs Net Margin compares profit efficiency and return to shareholders.
- Debt/Equity ratio helps see which sectors use more borrowed money

03

PAGE 3

- This page focuses on stock price behavior.
- Monthly adjusted close trend shows how stock prices changed over time.
- Volatility score shows which stocks move more sharply.
- Trading volume trend shows investor activity in market.
- Price change tells monthly increase or decrease in stock value.

04

PAGE 4

- This page compares sectors instead of individual companies.
- Sector-wise average net income shows profitable sectors.
- Revenue trend by sector shows sector growth over time.
- Profit margin by sector shows which sectors keep more profit.
- Debt burden by sector shows financial pressure across industries.

Part 3 : Python

Python Workflow Overview

EDA on

- Adjusted stock prices
- Financial metrics over years
- Sector-wise patterns

Time-Series Dataset Creation

- Monthly average adjusted closing price
- Monthly revenue (using fundamentals)

Trend & Seasonality

Forecast next 12 months of adjusted price for top 5 companies.

Category-Level Forecasting

Revenue forecast by GICS Sector.

```python
# Import All Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[67] ✓ 0.0s Python

```python
# Load Data
fundamentals = pd.read_csv(r"C:\Users\Divya Sharma\OneDrive\Desktop\Capstone_NY\fundamentals.csv")
Price_split_adj = pd.read_csv(r"C:\Users\Divya Sharma\OneDrive\Desktop\Capstone_NY\prices-split-adjusted.csv")
securies = pd.read_csv(r"C:\Users\Divya Sharma\OneDrive\Desktop\Capstone_NY\securities.csv")
```

[68] ✓ 0.4s Python

```python
# Columns
print(fundamentals.columns)
print(Price_split_adj.columns)
print(securies.columns)
```

[69] ✓ 0.0s Python

```
Index(['Unnamed: 0', 'Ticker Symbol', 'Period Ending', 'Accounts Payable',
       'Accounts Receivable', 'Add'l income/expense items', 'After Tax ROE',
       'Capital Expenditures', 'Capital Surplus', 'Cash Ratio',
       'Cash and Cash Equivalents', 'Changes in Inventories', 'Common Stocks',
       'Cost of Revenue', 'Current Ratio', 'Deferred Asset Charges',
       'Deferred Liability Charges', 'Depreciation',
       'Earnings Before Interest and Tax', 'Earnings Before Tax',
       'Effect of Exchange Rate',
       'Equity Earnings/Loss Unconsolidated Subsidiary', 'Fixed Assets',
       'Goodwill', 'Gross Margin', 'Gross Profit', 'Income Tax',
       'Intangible Assets', 'Interest Expense', 'Inventory', 'Investments',
       'Liabilities', 'Long-Term Debt', 'Long-Term Investments',
       'Minority Interest', 'Misc. Stocks', 'Net Borrowings', 'Net Cash Flow',
       'Net Cash Flow-Operating', 'Net Cash Flows-Financing',
       'Net Cash Flows-Investing', 'Net Income', 'Net Income Adjustments',
       'Net Income Applicable to Common Shareholders',
       'Net Income-Cont. Operations', 'Net Receivables', 'Non-Recurring Items',
       'Operating Income', 'Operating Margin', 'Other Assets',
       'Other Current Assets', 'Other Current Liabilities', 'Other Equity',
       'Other Financing Activities', 'Other Investing Activities',
       'Other Liabilities', 'Other Operating Activities',
       'Other Operating Items', 'Pre-Tax Margin', 'Pre-Tax ROE',
       'Profit Margin', 'Quick Ratio', 'Research and Development',
       'Retained Earnings', 'Sale and Purchase of Stock',
       'Sales, General and Admin.',
...
Index(['Ticker symbol', 'Security', 'SEC filings', 'GICS Sector',
       'GICS Sub Industry', 'Address of Headquarters', 'Date first added',
       'CIK'],
      dtype='str')
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Import Libraries

Load Datasets

Inspect Data

columns()

```python
# Check data
Price_split_adj.head()
fundamentals.head()
securies.head()
```

[70] ✓ 0.0s Python

	Ticker symbol	Security	SEC filings	GICS Sector	GICS Sub Industry	Address of Headquarters	Date first added	CIK
0	MMM	3M Company	reports	Industrials	Industrial Conglomerates	St. Paul, Minnesota	NaN	66740
1	ABT	Abbott Laboratories	reports	Health Care	Health Care Equipment	North Chicago, Illinois	1964-03-31	1800
2	ABBV	AbbVie	reports	Health Care	Pharmaceuticals	North Chicago, Illinois	2012-12-31	1551152
3	ACN	Accenture plc	reports	Information Technology	IT Consulting & Other Services	Dublin, Ireland	2011-07-06	1467373
4	ATVI	Activision Blizzard	reports	Information Technology	Home Entertainment Software	Santa Monica, California	2015-08-31	718877

```python
Price_split_adj.info()

fundamentals.info()

securies.info()
```

[71] ✓ 0.0s Python

```
<class 'pandas.DataFrame'>
RangeIndex: 851264 entries, 0 to 851263
Data columns (total 7 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   date    851264 non-null  str
 1   symbol  851264 non-null  str
 2   open    851264 non-null  float64
 3   close   851264 non-null  float64
 4   low     851264 non-null  float64
 5   high    851264 non-null  float64
 6   volume  851264 non-null  float64
dtypes: float64(5), str(2)
memory usage: 45.5 MB
<class 'pandas.DataFrame'>
RangeIndex: 1781 entries, 0 to 1780
Data columns (total 79 columns):
 #   Column                                          Non-Null Count  Dtype
---  ------                                          --------------  -----
 0   Unnamed: 0                                      1781 non-null   int64
 1   Ticker Symbol                                   1781 non-null   str
 2   Period Ending                                   1781 non-null   str
 3   Accounts Payable                                1781 non-null   float64
 4   Accounts Receivable                             1781 non-null   float64
 5   Add'l income/expense items                      1781 non-null   float64
...
 6   Date first added         307 non-null    str
 7   CIK                      505 non-null    int64
dtypes: int64(1), str(7)
memory usage: 31.7 KB
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

head()

info()

```python
# Converting Str dates in Datetime
Price_split_adj['date'] = pd.to_datetime(Price_split_adj['date'])
fundamentals['Period Ending'] = pd.to_datetime(fundamentals['Period Ending'])
securies['Date first added'] = pd.to_datetime(securies['Date first added'])
```

[72] ✓ 0.0s Python

```python
# Year extract
fundamentals['Year'] = fundamentals['Period Ending'].dt.year
```

[73] ✓ 0.0s Python

```python
# Remove unnecessary column
fundamentals.drop('Unnamed: 0', axis=1, inplace=True)
```

[74] ✓ 0.0s Python

PART 1 — EDA

```python
# 1 ADJUSTED STOCK PRICES
top10_symbols = Price_split_adj.groupby("symbol")["close"].mean().sort_values(ascending=False).head(10).index


top_10_Prices = Price_split_adj[Price_split_adj["symbol"].isin(top10_symbols)]

# Plot Adjusted Closing Prices (Top 10 Companies)

plt.figure(figsize=(14,7))

sns.lineplot(
    data = top_10_Prices,
    x='date',
    y='close',
    hue='symbol'
)


plt.title('Adjusted Stock Closing Prices')
plt.xlabel('Year')
plt.ylabel('Adjusted Closing Price')

plt.grid(True)

plt.show()
```

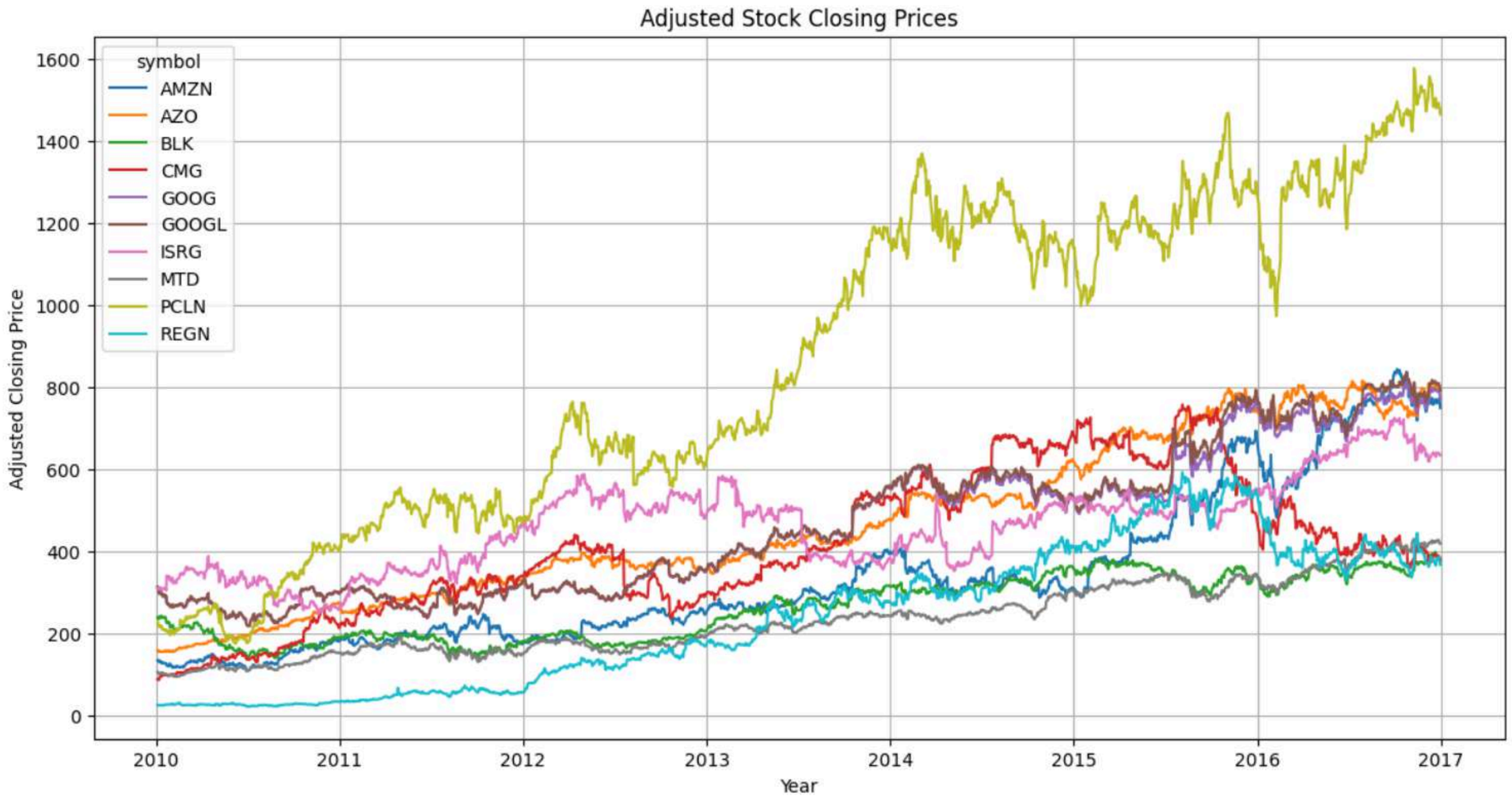
[108] ✓ 0.3s Python

Convert Date Columns

Only Extract Years

Remove Column

Part 1

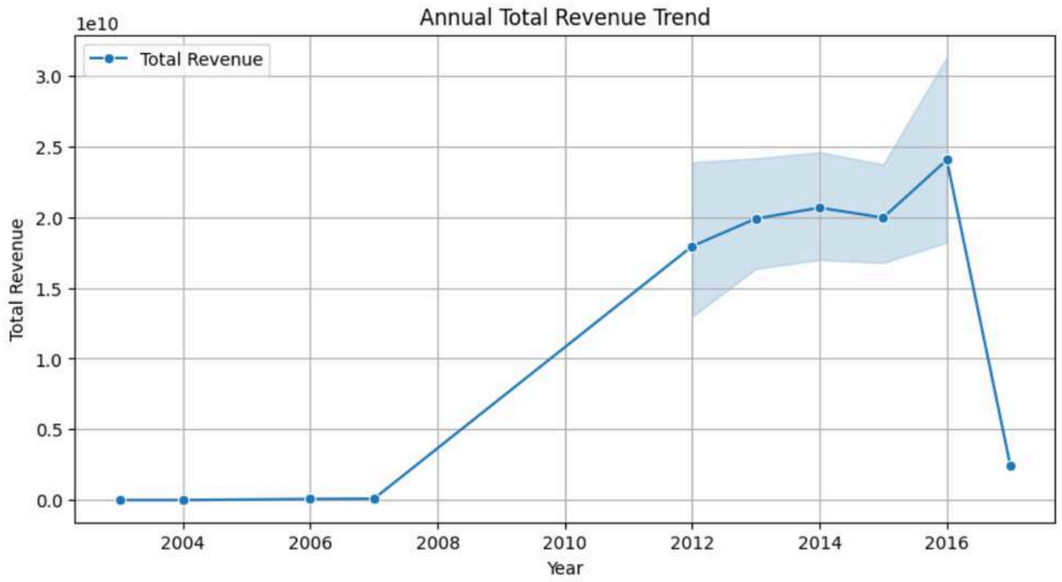



```python
# ADJ CLOSING PRICE TREND OF 'AMZN' (One Company)

apple = Price_split_adj[Price_split_adj['symbol'] == 'AMZN']
plt.figure(figsize=(10,5))
plt.plot(apple['date'], apple['close'])
plt.title("AMZN Adjusted Stock Closing Price")

plt.grid(True)
plt.show()
```

Python

```python
# 2 Financial Metrics over Years
plt.figure(figsize=(10,5))

sns.lineplot(
    data = fundamentals,
    x='Year',
    y='Total Revenue',
    marker="o",
    label="Total Revenue"
)

plt.title('Annual Total Revenue Trend')

plt.xlabel('Year')
plt.ylabel('Total Revenue')

plt.grid(True)

plt.show()
```

[110] ✓ 0.1s Python

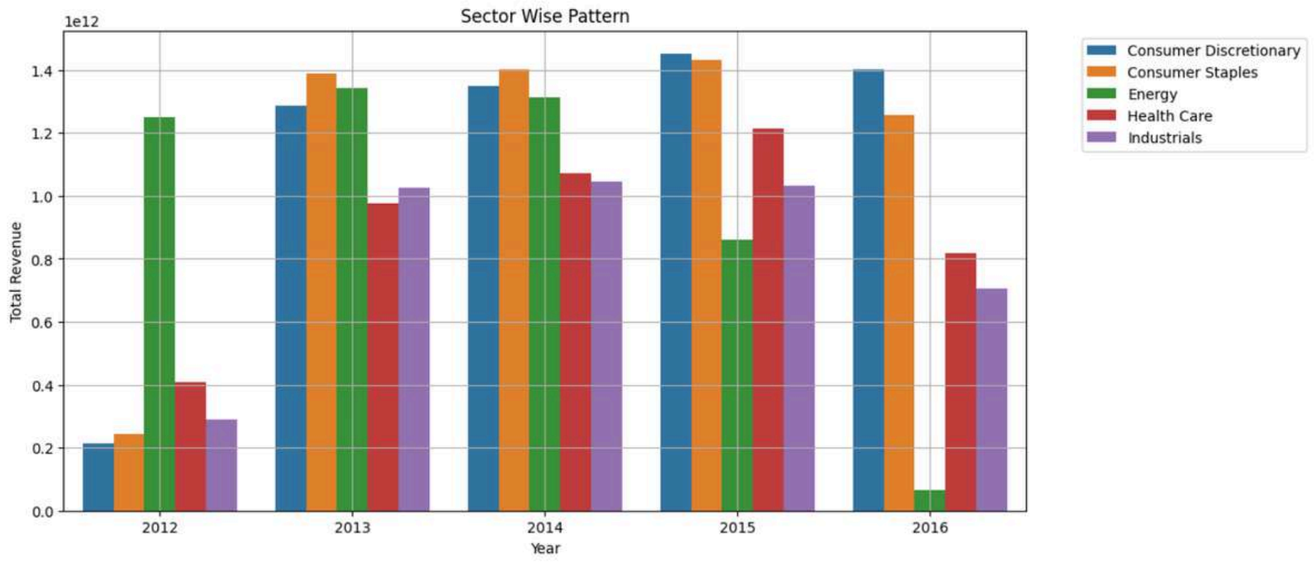



```python
#3 Sector-wise Patterns

# First we merge sector
fund_sec = fundamentals.merge(securies, left_on="Ticker Symbol", right_on="Ticker symbol", how="left")

# Filter valid years
fund_sect = fund_sec[fund_sec["Year"].between(2012, 2016)]

# Sector revenue (Group by Sector and Year)
sector_rev = fund_sect.groupby(["Year", "GICS Sector"])["Total Revenue"].sum().reset_index()

# # Groupby & Top 5
top_sectors = (sector_rev.groupby("GICS Sector")["Total Revenue"].sum().nlargest(5).index)

# Filter top sectors
sector_top = sector_rev[sector_rev["GICS Sector"].isin(top_sectors)]

# Convert year to string for clean axis
sector_rev["Year"] = sector_rev["Year"].astype(str)

# Plot - Total Revenue grouped by GICS Sector over years

plt.figure(figsize=(12,6))

sns.barplot(
    data=sector_top,
    x="Year",
    y="Total Revenue",
    hue="GICS Sector"
)
```

Python

```python
plt.title('Sector Wise Pattern')
plt.grid(True)

plt.legend(bbox_to_anchor=(1.05,1), loc="upper left")

plt.show()
```

Python



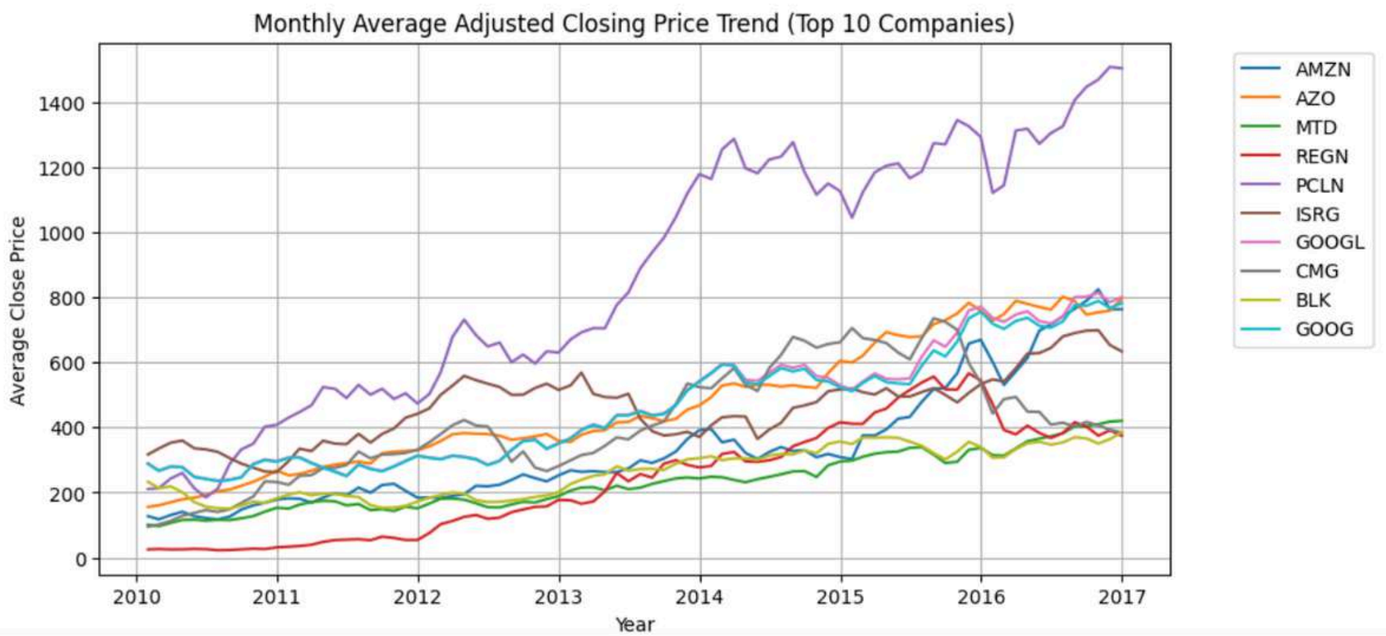
Part 2 Time series data creation

```python
# 1 Monthly average adjusted closing prices

# Create Monthly Averages
monthly_price = Price_split_adj.groupby(['symbol', pd.Grouper(key='date', freq='ME')])['close'].mean().reset_index()

# Identify Top 10 Companies
top10 = Price_split_adj.groupby('symbol')['close'].mean().sort_values(ascending=False).head(10).index

# Filter Monthly Data for Top 10
top10_prices = monthly_price[monthly_price['symbol'].isin(top10)]
top10_prices = top10_prices.sort_values('date')

plt.figure(figsize=(10,5))

sns.lineplot(
    data=top10_prices,
    x='date',
    y='close',
    hue='symbol'
)

plt.title('Monthly Average Adjusted Closing Price Trend (Top 10 Companies)')

plt.xlabel('Year')
plt.ylabel('Average Close Price')
```

Part 2

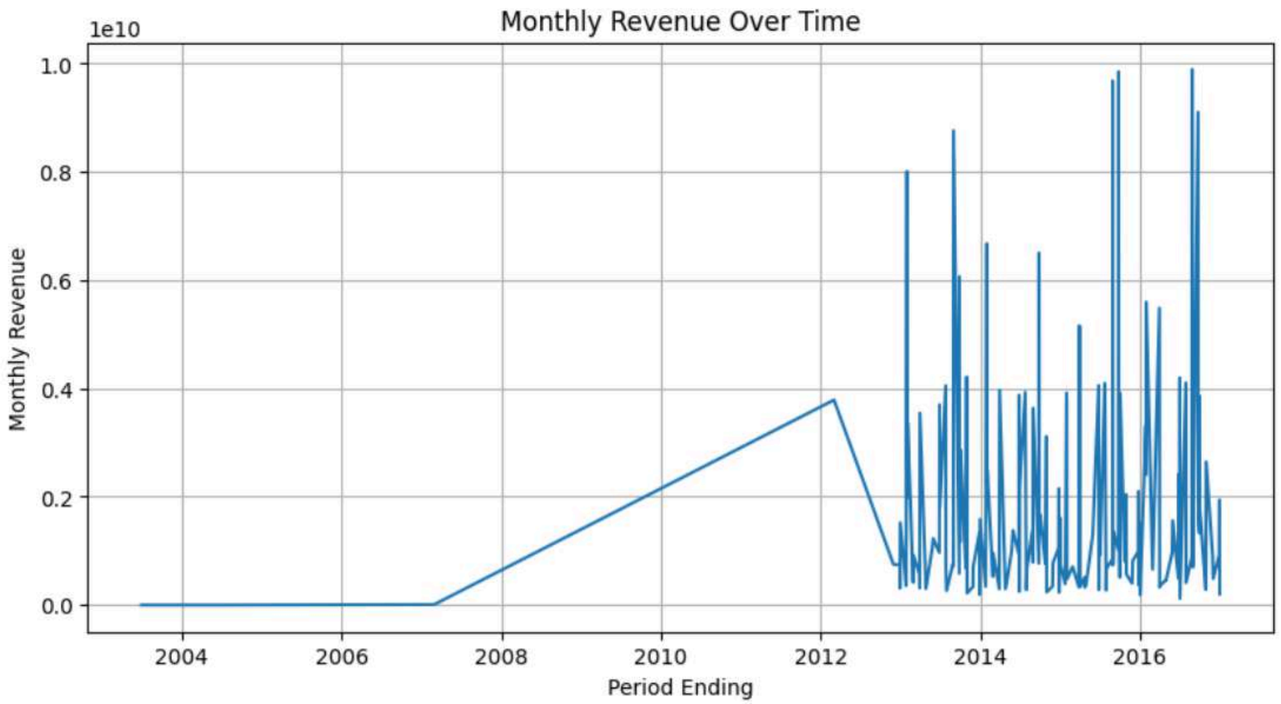
```python
plt.legend(bbox_to_anchor=(1.05,1), loc='upper left')

plt.grid(True)

plt.show()
```

Python




```python
# 2.  Monthly Revenue (Using Fundamentals)

fundamentals["monthly_revenue"] = fundamentals["Total Revenue"] / 12

monthly_revenue = fundamentals.groupby( "Period Ending")["monthly_revenue"].mean().reset_index()

plt.figure(figsize=(10, 5))

sns.lineplot(data=monthly_revenue,
             x="Period Ending",
             y="monthly_revenue")

plt.title("Monthly Revenue Over Time")

plt.xlabel("Period Ending")

plt.ylabel("Monthly Revenue")

plt.grid(True)
plt.show()
```

Python

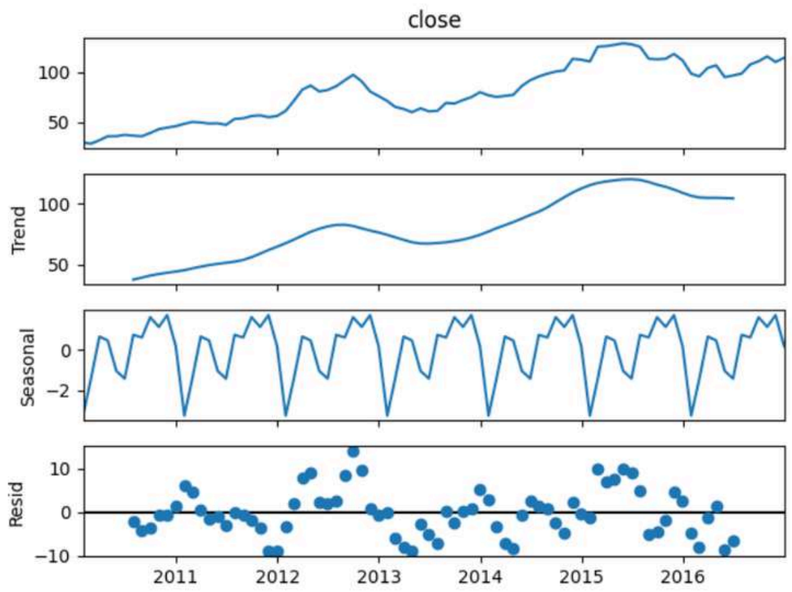


Part 3 Trend and Seasonality

```python
# 1. Use statsmodels or Prophet to detect seasonality in adjusted close prices - Used Statsmodels

# Select One Company (Original Price)
aapl = monthly_price[monthly_price['symbol']=='AAPL']

# set date as Index
aapl = aapl.set_index('date')

# Import Deccomposition
from statsmodels.tsa.seasonal import seasonal_decompose

# Apply Decomposition
result = seasonal_decompose(aapl['close'],model='additive',period=12)

result.plot()

plt.show()
```

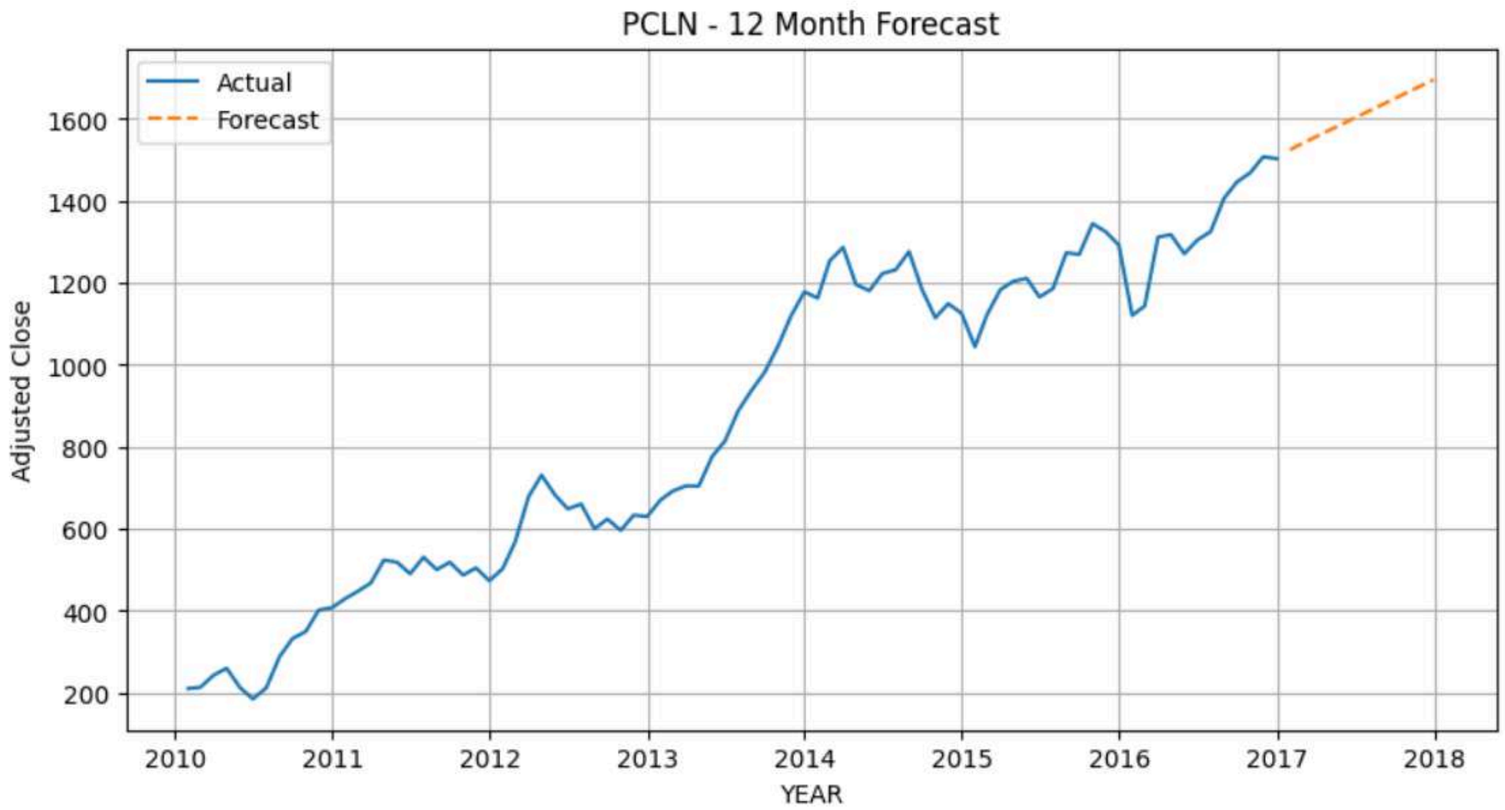
Python

Part 3

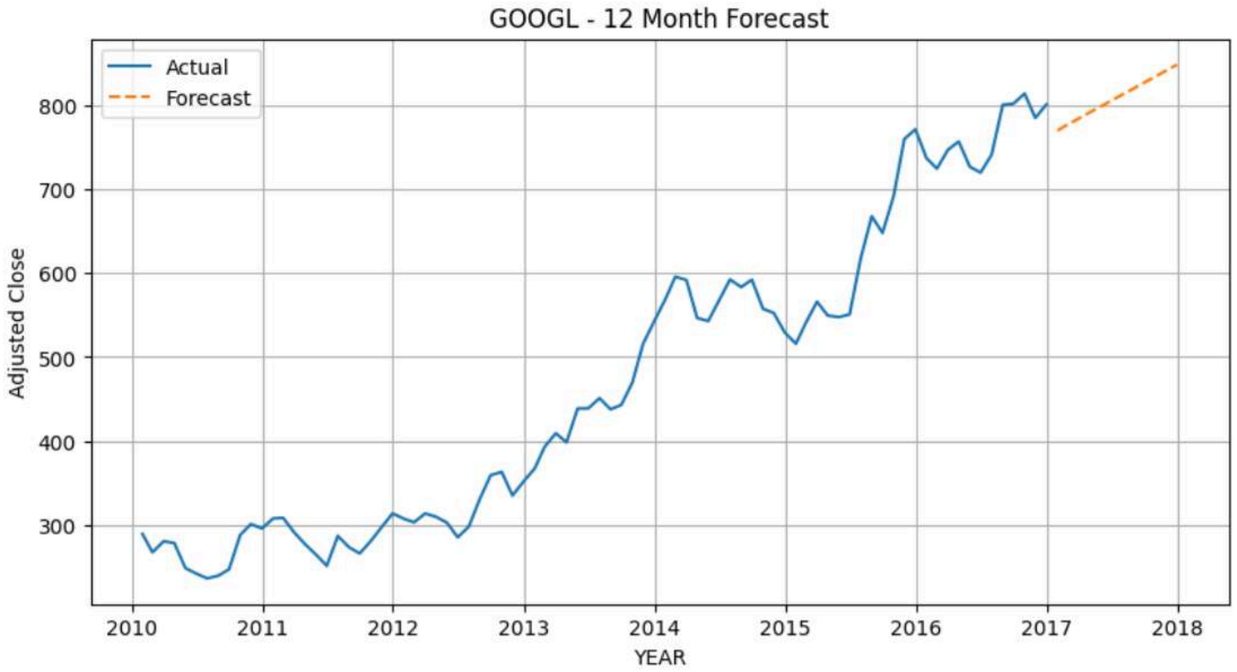



```python
# 2. Forecast next 12 months of adjusted price for top 5 companies - Used LinearRegression

from sklearn.linear_model import LinearRegression

top5 = (Price_split_adj.groupby('symbol')['close'].mean().sort_values(ascending=False).head(5).index)

for company in top5:

    company_data = monthly_price[monthly_price['symbol'] == company].copy()

    company_data = company_data.sort_values('date')

    company_data['Month_Num'] = range(len(company_data))

    X = company_data[['Month_Num']]
    y = company_data['close']

    model = LinearRegression()
    model.fit(X, y)

    future_months = pd.DataFrame({
        'Month_Num': range(len(company_data), len(company_data)+12)
    })

    forecast = model.predict(future_months)

    future_dates = pd.date_range(
        start=company_data['date'].iloc[-1] + pd.offsets.MonthEnd(1),
        periods=12,
        freq='ME'
```

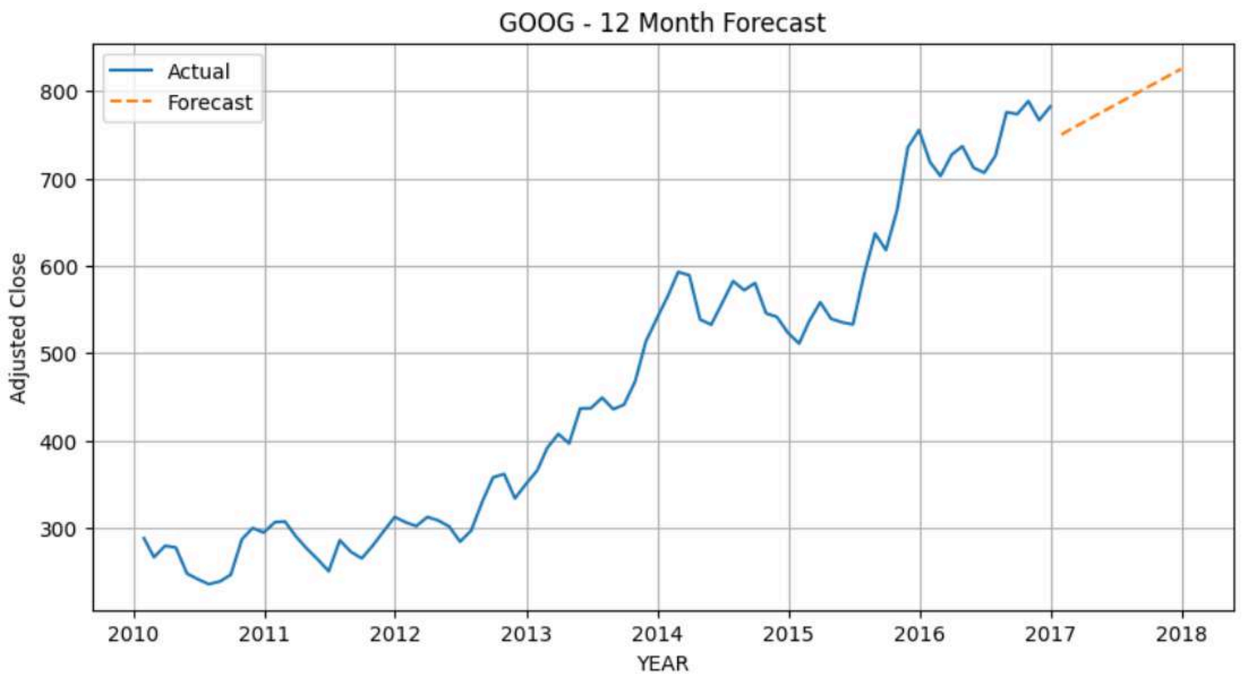
```python
)

plt.figure(figsize=(10,5))

plt.plot(company_data['date'], company_data['close'], label='Actual')
plt.plot(future_dates, forecast, label='Forecast', linestyle="--")

plt.title(f'{company} - 12 Month Forecast')
plt.xlabel('YEAR')
plt.ylabel('Adjusted Close')

plt.legend()
plt.grid(True)

plt.show()
```



AZO - 12 Month Forecast
Actual
Forecast
Adjusted Close
900
800
700
600
500
400
300
200
2010
2011
2012
2013
2014
2015
2016
2017
2018
YEAR

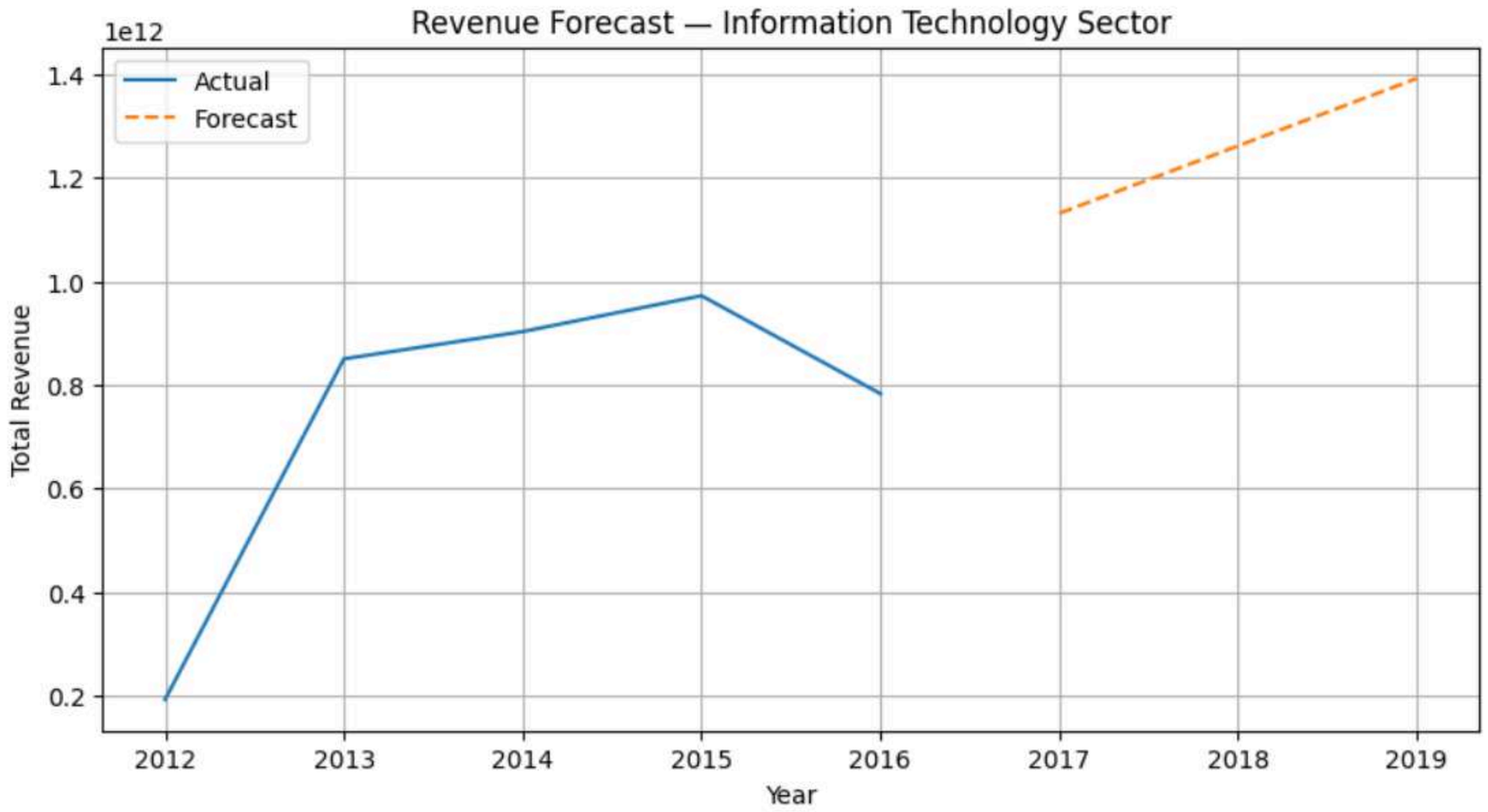

GOOGL - 12 Month Forecast
Actual
Forecast
Adjusted Close
800
700
600
500
400
300
2010
2011
2012
2013
2014
2015
2016
2017
2018
YEAR

ISRG - 12 Month Forecast
Actual
Forecast
Adjusted Close
700
600
500
400
300
2010
2011
2012
2013
2014
2015
2016
2017
2018
YEAR


GOOG - 12 Month Forecast
Actual
Forecast
Adjusted Close
800
700
600
500
400
300
2010
2011
2012
2013
2014
2015
2016
2017
2018
YEAR

Part 4 - Category-Level Forecasting

```python
# 1. Revenue forecast by GICS Sector

from sklearn.linear_model import LinearRegression

sector_data = fundamentals.merge(
    securies[['Ticker symbol', 'GICS Sector']],
    left_on='Ticker Symbol',
    right_on='Ticker symbol',
    how='left'
)

sector_data['Year'] = sector_data['Period Ending'].dt.year.astype(int)

sector_rev = (
    sector_data.groupby(['GICS Sector', 'Year'])['Total Revenue']
    .sum()
    .reset_index()
)

it_sector = sector_rev[
    sector_rev['GICS Sector'] == 'Information Technology'
].copy()

it_sector['Year_Num'] = range(len(it_sector))

X = it_sector[['Year_Num']]
y = it_sector['Total Revenue']
model = LinearRegression()
model.fit(X, y)

future_years = pd.DataFrame({
    'Year_Num': range(len(it_sector), len(it_sector)+3)
})

forecast = model.predict(future_years)

future_labels = range(
    it_sector['Year'].max()+1,
    it_sector['Year'].max()+4
)

plt.figure(figsize=(10,5))

plt.title('Revenue Forecast — Information Technology Sector')
plt.xlabel('Year')
plt.ylabel('Total Revenue')

plt.plot(it_sector['Year'], it_sector['Total Revenue'], label='Actual')
plt.plot(future_labels, forecast, label='Forecast', linestyle="--")

plt.legend()
plt.grid(True)

plt.show()
```


Revenue Forecast — Information Technology Sector
1e12
Actual
Forecast
Total Revenue
1.4
1.2
1.0
0.8
0.6
0.4
0.2
2012
2013
2014
2015
2016
2017
2018
2019
Year

28930.76
28930.43
28930.42
28929.91
28929.89
28929.88
28928.62
28927.75
28926.29
28925.75
28925.73
28925.11
28925.08
28923.39
28925.75

Executive Summary

PROJECT OBJECTIVE

- ANALYZED COMPANY FUNDAMENTALS AND STOCK PRICE DATA FROM THE NYSE.
- FOCUSED ON SECTOR PERFORMANCE, VALUATION, AND FINANCIAL HEALTH.
- EVALUATED STOCK BEHAVIOR FOR INVESTMENT UNDERSTANDING.
- GENERATED BUSINESS INSIGHTS FOR DECISION-MAKING.

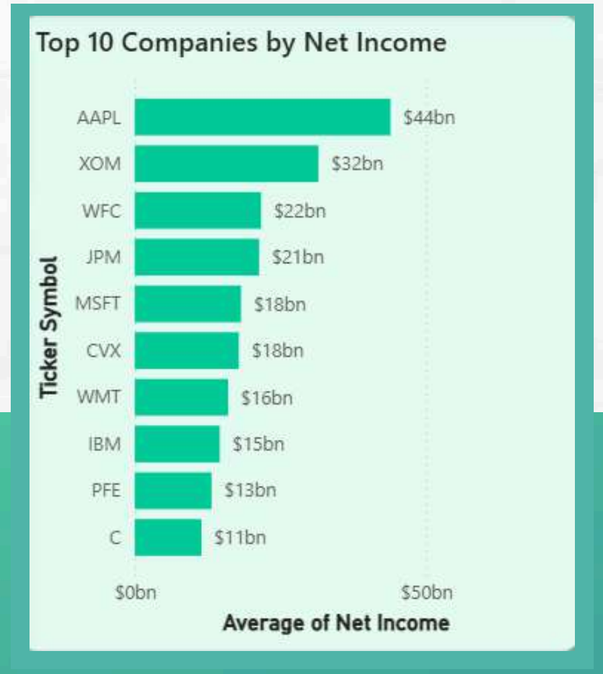
TOOLS & ANALYSIS PERFORMED

- USED SQL FOR FINANCIAL QUERIES AND DATA EXTRACTION.
- BUILT DASHBOARD IN MICROSOFT POWER BI FOR INTERACTIVE ANALYSIS.
- APPLIED PYTHON FOR TREND, VOLATILITY, AND FORECASTING.
- COMBINED MULTIPLE DATASETS FOR COMPLETE MARKET EVALUATION.

KEY OUTCOMES

- IDENTIFIED STRONG SECTORS WITH STABLE FINANCIAL PERFORMANCE.
- DETECTED VALUATION DIFFERENCES ACROSS COMPANIES.
- HIGHLIGHTED POTENTIAL UNDERVALUED STOCKS.
- PROVIDED RECOMMENDATIONS FOR LONG-TERM INVESTMENT FOCUS.

Visuals Highlights From Dashboard

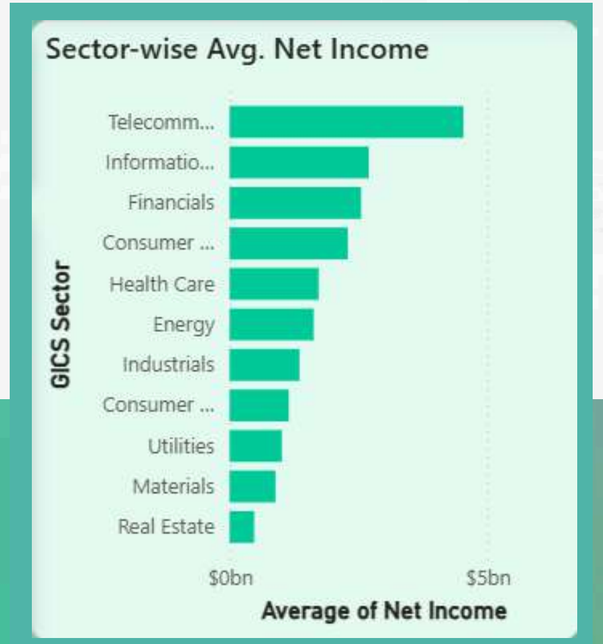


PROFITABILITY CHART

Top 10 Companies by Net Income highlights the highest earning companies and shows which firms generate stronger profits.

STOCK TREND CHART

Monthly Adjusted Closing Price Trends tracks historical stock movement and helps understand trend direction and volatility.



SECTOR COMPARISON CHART

Sector-wise Average Net Income compares profitability across sectors and helps identify industries with stronger long-term financial performance.

Trends, Anomalies And Opportunities

TRENDS

Technology and Healthcare sectors show stronger profitability and more stable long-term financial performance compared to other sectors.

ANOMALIES

Some companies have high revenue but lower net income, indicating weaker profit conversion despite strong sales.

OPPORTUNITIES

Stocks with lower P/E ratio, positive EPS, and stable earnings may represent potential undervalued investment opportunities.

Actionable Recommendations:

WHICH SECTOR SHOWS LONG-TERM HEALTH?

Technology and Healthcare sectors appear stronger for long-term investment due to stable profitability and consistent earnings performance.

WHICH STOCKS ARE OVER/UNDERVALUED?

Stocks with low P/E ratio and positive EPS may be undervalued, while high P/E with weaker earnings may indicate overvaluation.

WHERE TO INVEST BASED ON PRICE TREND + FUNDAMENTALS?

Based on financial fundamentals and historical price trends, stocks such as AAPL, MSFT, and JNJ can be considered stronger investment candidates due to stable earnings, healthy profitability, and consistent market performance.

MySQL

Power BI

python™

Thank You

LinkedIn	www.linkedin.com/in/divya9916
GitHub	github.com/Divya9916